

CRUD e Consultas no MongoDB — Aula 05/10

Professora Gislene

1. CREATE – Inserindo Dados

1.1 Inserir um único documento

Documento com os campos básicos de um livro. O `_id` é gerado automaticamente pelo MongoDB.

json

```
{
  "titulo": "Dom Casmurro",
  "autor": "Machado de Assis",
  "ano": 1899,
  "genero": "Romance",
  "paginas": 256,
  "disponivel": true
}
```

Comentário campo a campo:

Campo	Valor	Tipo	Observação
titulo	"Dom Casmurro"	String	Nome do livro
autor	"Machado de Assis"	String	Nome do autor
ano	1899	Número	Ano de publicação
genero	"Romance"	String	Categoria do livro
paginas	256	Número	Quantidade de páginas
disponivel	true	Booleano	Indica se está disponível

1.2 Inserir vários documentos (array)

Array com três documentos para inserir de uma só vez. Muito mais eficiente que inserir um por um.

json

```
[
  {
```

```

    "titulo": "1984",
    "autor": "George Orwell",
    "ano": 1949,
    "genero": "Distopia",
    "paginas": 416,
    "disponivel": true
  },
  {
    "titulo": "O Hobbit",
    "autor": "J.R.R. Tolkien",
    "ano": 1937,
    "genero": "Fantasia",
    "paginas": 310,
    "disponivel": false
  },
  {
    "titulo": "Sapiens",
    "autor": "Yuval Harari",
    "ano": 2011,
    "genero": "História",
    "paginas": 464,
    "disponivel": true
  }
]

```

Comentário:

- Cada objeto do array é um documento independente.
 - O MongoDB cria automaticamente um `_id` único para cada um.
 - `disponivel: false` no "O Hobbit" mostra que o campo é booleano e pode variar por documento.
-

1.3 Documento com subdocumentos (aninhamento)

Mostra como o MongoDB permite aninhar objetos e arrays dentro de um documento, reduzindo a necessidade de JOINS.

json

```

{
  "titulo": "Clean Code",
  "autor": {
    "nome": "Robert Martin",
    "nacionalidade": "EUA"
  },
  "editora": {
    "nome": "Prentice Hall",
    "ano": 2008
  },
  "tags": ["programação", "boas práticas"],
  "disponivel": true
}

```

```
}
```

Comentário:

Parte	O que é	Como acessar depois
autor	Subdocumento com nome e nacionalidade	autor.nome
editora	Subdocumento com nome e ano	editora.ano
tags	Array de strings	tags ou tags.0

2. READ – Consultas (Filter do Compass)

2.1 Filtrar por um campo

Equivalente ao WHERE genero = "Fantasia" do SQL. Retorna todos os documentos cujo gênero seja "Fantasia".

json

```
{ "genero": "Fantasia" }
```

Comentário: a chave é o campo e o valor é o que se busca. É um match exato.

2.2 Filtrar por múltiplos campos (AND implícito)

Quando há mais de um campo no mesmo objeto, o MongoDB aplica AND automaticamente.

json

```
{ "genero": "Fantasia", "disponivel": true }
```

Comentário: retorna documentos em que genero = "Fantasia" E disponível = true ao mesmo tempo.

2.3 Projeção (segundo argumento do find)

Define quais campos devem aparecer no resultado.

json

```
{ "titulo": 1, "autor": 1, "_id": 0 }
```

Comentário:

Valor	Significado
1	Incluir o campo no resultado
0	Excluir o campo do resultado
_id: 0	Ocultar o _id (que vem por padrão)
No Compass, use a aba Project. No mongosh, é o segundo argumento de find().	

2.4 Ordenação (sort)

Ordena os resultados por um ou mais campos.

json

```
{ "ano": 1 }
```

Comentário: 1 = ordem ascendente (do mais antigo para o mais novo).

json

```
{ "ano": -1 }
```

Comentário: -1 = ordem descendente (do mais novo para o mais antigo).

3. UPDATE – Atualizações

O \$ (cifrão) é um prefixo reservado do MongoDB que indica "isto é um operador, não um campo comum". Ele resolve um problema fundamental: como diferenciar um campo de dados de um comando.

3.1 Atualizar um campo (\$set)

Define um novo valor para um campo. Cria o campo se ele ainda não existir.

json

```
{  
  "$set": { "disponivel": false }  
}
```

Comentário: altera o campo disponivel para false no(s) documento(s) que casarem com o filtro.

3.2 Atualizar vários documentos

Mesmo operador \$set, mas aplicado a todos os documentos que casarem com o filtro.

json

```
{
  "$set": { "destaque": true }
}
```

Comentário: adiciona um campo novo destaque com valor true em todos os documentos correspondentes.

3.3 Incrementar valor (\$inc)

Soma (ou subtrai, com valor negativo) um valor a um campo numérico.

```
json
{
  "$inc": { "paginas": 10 }
}
```

Comentário: soma 10 ao valor atual de paginas. Se paginas fosse 256, passaria a 266.

3.4 Renomear campo (\$rename)

Troca o nome de um campo, mantendo o valor.

```
json
{
  "$rename": { "ano": "anoPublicacao" }
}
```

Comentário: o campo ano vira anoPublicacao com o mesmo valor.

3.5 Remover campo (\$unset)

Remove definitivamente um campo do documento. O valor passado ("") é ignorado.

```
json
{
  "$unset": { "destaque": "" }
}
```

Comentário: o campo destaque é apagado. O restante do documento permanece intacto.

3.6 Adicionar item em array (\$push)

Adiciona um valor ao final do array.

```
json
{
  "$push": { "tags": "clássico" }
}
```

Comentário: insere "clássico" no array tags. Se o item já existir, ele é duplicado.

3.7 Adicionar sem duplicar (\$addToSet)

Adiciona um valor ao array apenas se ele ainda não existir.

```
json
{
  "$addToSet": { "tags": "clássico" }
}
```

Comentário: se "clássico" já estiver em tags, nada acontece.

3.8 Remover item de array (\$pull)

Remove todas as ocorrências de um valor no array.

```
json
{
  "$pull": { "tags": "clássico" }
}
```

Comentário: apaga "clássico" do array tags (todas as ocorrências).

3.9 Substituir documento inteiro (replaceOne – corpo)

Substitui o documento inteiro, mantendo apenas o _id.

```
json
{
  "titulo": "Sapiens",
  "autor": "Yuval Harari",
  "ano": 2015,
  "disponivel": true
}
```

Comentário: campos antigos que não estiverem aqui serão perdidos.

3.10 Upsert (inserir se não existir)

Combina \$set com a opção { "upsert": true } para criar o documento caso ele não exista.

```
json
{
  "$set": { "autor": "Autor X", "ano": 2024 }
}
```

Comentário: use junto com a opção { "upsert": true }.

- Se encontrar o documento → atualiza.
 - Se não encontrar → insere um novo.
-

4. DELETE – Remoções (Filter do Compass)

4.1 Remover um documento

Remove apenas o primeiro documento que casar com o filtro.

```
json
{ "titulo": "1984" }
```

Comentário: no mongosh usa-se `deleteOne(<filtro>)`. No Compass, use o ícone de lixeira com este filtro.

4.2 Remover vários

Remove todos os documentos que casarem com o filtro.

```
json
{ "disponivel": false }
```

Comentário: no mongosh, use `deleteMany(<filtro>)`.

4.3 Remover todos (filtro vazio)

Remove todos os documentos da coleção. A coleção continua existindo, mas vazia.

```
json
{ }
```

Comentário: operação perigosa. Use com cautela. No mongosh, `deleteMany({})`.

5. Consultas Avançadas

5.1 Operadores de comparação

Maior que (`$gt`):

```
json
{ "ano": { "$gt": 1950 } }
```

Comentário: livros publicados após 1950.

Está na lista (`$in`):

```
json
{ "genero": { "$in": ["Fantasia", "Distopia"] } }
```

Comentário: livros cujo gênero seja "Fantasia" ou "Distopia".

Intervalo (

`gte+`

`gte+lte`):

```
json
{ "paginas": { "$gte": 300, "$lte": 500 } }
```

Comentário: livros com 300 a 500 páginas, inclusive.

Diferente (`$ne`):

json

```
{ "genero": { "$ne": "Romance" } }
```

Comentário: livros cujo gênero não é "Romance".

5.2 Operadores lógicos

\$and:

json

```
{
  "$and": [
    { "genero": "Fantasia" },
    { "ano": { "$lt": 2000 } }
  ]
}
```

Comentário: precisa atender todas as condições ao mesmo tempo.

\$or:

json

```
{
  "$or": [
    { "genero": "Fantasia" },
    { "genero": "Distopia" }
  ]
}
```

Comentário: precisa atender pelo menos uma das condições.

\$not:

json

```
{ "ano": { "$not": { "$gt": 2000 } } }
```

Comentário: inverte a lógica. Retorna livros que NÃO são posteriores a 2000.

\$nor:

json

```
{
  "$nor": [
    { "genero": "Romance" },
    { "disponivel": false }
  ]
}
```

Comentário: retorna documentos que não atendem nenhuma das condições.

5.3 Regex

Busca case-insensitive:

json

```
{ "titulo": { "$regex": "hobbit", "$options": "i" } }
```

Comentário: encontra títulos que contenham "hobbit" em qualquer posição, ignorando maiúsculas/minúsculas.

Busca por prefixo:

json

```
{ "autor": { "$regex": "^George" } }
```

Comentário: autores cujo nome começa com "George".

5.4 Arrays

Contém valor:

json

```
{ "tags": "programação" }
```

Comentário: retorna documentos cujo array tags contenha o item "programação".

Contém todos (\$all):

json

```
{ "tags": { "$all": ["clássico", "fantasia"] } }
```

Comentário: o array tags precisa conter todos os itens informados.

Tamanho exato (\$size):

json

```
{ "tags": { "$size": 3 } }
```

Comentário: array tags com exatamente 3 itens.

5.5 Subdocumentos (notação de ponto)

json

```
{ "autor.nome": "Robert Martin" }
```

Comentário: acessa o campo nome dentro do subdocumento autor.

json

```
{ "editora.ano": { "$gt": 2000 } }
```

Comentário: editoras cujo ano de fundação seja após 2000.

5.6 Existência de campo

json

```
{ "destaque": { "$exists": true } }
```

Comentário: retorna documentos que possuem o campo destaque.

json

```
{ "destaque": { "$exists": false } }
```

Comentário: retorna documentos que não possuem o campo destaque.

6. Aggregation Pipeline (JSON)

O pipeline é um array de estágios. Cada estágio transforma os dados e passa para o próximo.

6.1 Pipeline básico

json

```
[
  { "$match": { "disponivel": true } },
  { "$group": { "_id": "$genero", "total": { "$sum": 1 } } },
  { "$sort": { "total": -1 } }
]
```

Comentário estágio por estágio:

Estágio	O que faz
\$match	Filtra apenas livros com disponivel: true
\$group	Agrupar por genero e conta com \$sum: 1
\$sort	Ordena do gênero com mais livros para o com menos
O \$ antes de genero indica "valor do campo genero".	

6.2 Média de páginas por gênero

```
json
[
  {
    "$group": {
      "_id": "$genero",
      "mediaPaginas": { "$avg": "$paginas" }
    }
  }
]
```

Comentário: agrupa por gênero e calcula a média do campo paginas de cada grupo.

6.3 Total de livros por autor

```
json
[
  {
    "$group": {
      "_id": "$autor",
      "total": { "$sum": 1 }
    }
  },
  { "$sort": { "total": -1 } },
  { "$limit": 5 }
]
```

Comentário:

Estágio	O que faz
\$group	Conta quantos livros cada autor tem
\$sort	Ordena do autor com mais livros para o com menos
\$limit	Mostra apenas os top 5

6.4 JOIN com \$lookup

json

```
[
  {
    "$lookup": {
      "from": "autores",
      "localField": "autor",
      "foreignField": "nome",
      "as": "infoAutor"
    }
  }
]
```

Comentário campo a campo:

Campo	Significado
from	Coleção de onde buscar os dados (autores)
localField	Campo no documento atual (autor)
foreignField	Campo correspondente na outra coleção (nome)
as	Nome do array resultante com os dados juntados (infoAutor)

Equivale a um LEFT JOIN do SQL.

6.5 Desmembrar arrays com \$unwind

json

```
[
  { "$unwind": "$tags" },
  { "$group": { "_id": "$tags", "total": { "$sum": 1 } } },
  { "$sort": { "total": -1 } }
]
```

]

Comentário:

Estágio	O que faz
\$unwind	Cria um documento para cada item do array tags
\$group	Agrupar por tag e conta ocorrências
\$sort	Ordena das tags mais frequentes para as menos

6.6 Transformar campos com \$project

json

```
[
  {
    "$project": {
      "_id": 0,
      "titulo": 1,
      "autorMaiusculo": { "$toUpper": "$autor" },
      "decada": { "$subtract": [ "$ano", { "$mod": [ "$ano", 10 ] } ] }
    }
  }
]
```

Comentário campo a campo:

Campo	O que faz
_id: 0	Ocultar o _id
titulo: 1	Mantém o titulo
autorMaiusculo	Cria campo novo com autor em maiúsculas (\$toUpper)
decada	Cria campo novo calculando a década: ano - (ano % 10)

6.7 Contar com \$count

json

```
[
  { "$match": { "disponivel": true } },
```

```
{ "$count": "totalDisponiveis" }  
]
```

Comentário: filtra os disponíveis e conta. Retorna { "totalDisponiveis": N }.

7. Índices

7.1 Índice simples

```
json  
{ "titulo": 1 }
```

Comentário: índice ascendente no campo titulo. Acelera buscas e ordenações por esse campo.

7.2 Índice composto

```
json  
{ "autor": 1, "ano": -1 }
```

Comentário: índice com dois campos. A ordem importa: primeiro por autor (asc), depois por ano (desc).

7.3 Índice de texto

```
json  
{ "titulo": "text", "autor": "text" }
```

Comentário: habilita busca textual com \$text nesses campos. Aceita busca por múltiplas palavras e ignora stopwords.

7.4 Índice TTL

```
json  
{ "expira_em": 1 }
```

Comentário: use junto com a opção { "expireAfterSeconds": 0 } no createIndex. O MongoDB apaga automaticamente os documentos após o tempo definido no campo expira_em.

8. Projeto Final – Sistema de Biblioteca

8.1 Documentos da coleção livros

```
json  
[  
  { "_id": 1, "titulo": "Dom Casmurro", "autor": "Machado de Assis", "ano": 1899, "disponivel":  
true },  
  { "_id": 2, "titulo": "1984", "autor": "George Orwell", "ano": 1949, "disponivel": true },
```

```
{ "_id": 3, "titulo": "O Hobbit", "autor": "Tolkien", "ano": 1937, "disponivel": false }
]
```

Comentário: `_id` definido manualmente (1, 2, 3) para facilitar os relacionamentos com as outras coleções.

8.2 Documentos da coleção usuarios

```
json
[
  { "_id": 1, "nome": "Ana", "email": "ana@email.com" },
  { "_id": 2, "nome": "Bruno", "email": "bruno@email.com" }
]
```

Comentário: `_id` numérico para casar com o campo `usuariold` dos empréstimos.

8.3 Documentos da coleção emprestimos

```
json
[
  { "livrold": 3, "usuariold": 1, "dataEmprestimo": "2025-01-10T00:00:00Z", "devolvido": false },
  { "livrold": 2, "usuariold": 2, "dataEmprestimo": "2025-01-15T00:00:00Z", "devolvido": false }
]
```

Comentário campo a campo:

Campo	O que representa
livrold	Referência ao <code>_id</code> de livros
usuariold	Referência ao <code>_id</code> de usuarios
dataEmprestimo	Data do empréstimo em ISO 8601
devolvido	Booleano indicando se já foi devolvido

⚠️ Atenção: No mongosh, `new Date("2025-01-10")` gera um `ISODate`. Em JSON puro (Compass), use string ISO 8601 ou o seletor de tipo `Date` do Compass.

8.4 Filtro – livros disponíveis

```
json
{ "disponivel": true }
```

Comentário: retorna apenas livros disponíveis para empréstimo.

8.5 Filtro – empréstimos de um usuário

json

```
{ "usuarioid": 1 }
```

Comentário: retorna todos os empréstimos feitos pelo usuário de _id: 1.

8.6 Atualizar livro como emprestado

json

```
{ "$set": { "disponivel": false } }
```

Comentário: marca o livro como indisponível após o empréstimo.

8.7 Remover empréstimos antigos (mais de 30 dias e já devolvidos)

json

```
{  
  "dataEmprestimo": { "$lt": "2024-12-10T00:00:00Z" },  
  "devolvido": true  
}
```

Comentário: filtro que remove empréstimos anteriores a 10/12/2024 E já devolvidos. No mongosh, ajuste a data dinamicamente.

8.8 Top 3 livros mais emprestados (aggregate)

json

```
[  
  { "$group": { "_id": "$livroid", "total": { "$sum": 1 } } },  
  { "$sort": { "total": -1 } },  
  { "$limit": 3 },  
  {  
    "$lookup": {  
      "from": "livros",  
      "localField": "_id",  
      "foreignField": "_id",  
      "as": "livro"  
    }  
  }  
]
```

Comentário estágio por estágio:

Estágio

O que faz

\$group

Agrupa por livroid e conta quantos empréstimos cada livro teve

\$sort	Ordena do mais emprestado para o menos
\$limit	Pega apenas o top 3
\$lookup	Junta com a coleção livros para trazer os dados completos do livro

8.9 Índices do projeto

json

```
{ "titulo": 1 }
```

Comentário: acelera buscas por título na coleção livros.

json

```
{ "usuarioid": 1 }
```

Comentário: acelera buscas de empréstimos por usuário.

json

```
{ "dataEmprestimo": 1 }
```

Comentário: acelera consultas por data de empréstimo (útil para relatórios e limpezas automáticas).

Como aplicar cada bloco JSON

Onde usar	O que colar
Compass → Filter	Blocos da seção 2 (READ) e 4 (DELETE)
Compass → Project	Bloco 2.3
Compass → Sort	Bloco 2.4
Compass → Update	Blocos da seção 3 (UPDATE)
Compass → Aggregations → Pipeline	Blocos da seção 6
Compass → Indexes → Create	Blocos da seção 7
mongosh	Use como argumento de find(), updateOne(), aggregate(), etc.

